

LAB 6 (SECJ1023)
PROGRAMMING TECHNIQUE II
SECTION 03, 04 & 06, SEM 2, 2025/2026

1. Given class declarations in the following code segment. Explain or draw the relationship between the class **Person**, **TextBook** and **Course**.

```
1 //Program 6.1
2 class Person
3 {
4     private:
5         string name;
6 };
7
8 class TextBook
9 {
10     private:
11         Person author;
12 };
13 class Course
14 {
15     private:
16         TextBook book;
17         Person instructor;
18 };
```

2. Consider the following Program 6.2. and answer questions (a), (b) and (c).

```
1 //Program 6.2
2 #include <iostream>
3 using namespace std;
4
5 class Officer {
6 private :
7     string name;
8     int id;
9
10 public :
11     Officer (string oname = "", int oid = 0) {
12         name = oname; id = oid;}
13
14     void set(string a, int b) {
15         name = a; id = b; }
16
17     void print() {
18         cout << "Officer    : " << name << " " << id << endl; }
19 };
20
21 class Offender {
22 private :
23     string name, ic, address;
24 }
```

```

25 public :
26     Offender(string oname="", string id="", string add=""){
27         name = oname; address = add; ic = id; }
28
29     void set(string a, string b, string c) {
30         name = a; ic = b; address = c;}
31
32     void print() {
33         cout << "Offender :" << name << " " << ic << " "
34             << address << endl; }
35 };
36
37 class Summon {
38 private :
39     int code;
40     Officer *officer;
41     Offender offender;
42
43 public :
44     Summon (int cod=0, string name="", string id="",
45         string add = "" ){
46         code = cod; offender.set(name,id,add); }
47
48     void addOfficer(string a, int b) {
49         officer = new Officer; officer->set(a,b); }
50
51     void removeOfficer() {
52         delete officer; officer = NULL; }
53
54     void print() {
55         cout<<endl<<"Summon Code : "<<code<<endl;
56         offender.print();
57         officer->print(); }
58 };
59
60 int main()
61 {
62     Officer a("ALI", 75381);
63     Offender b("KASSIM","841203015311",
64         "NO 1 JALAN PS 13 TAMAN PERMATA SINAR");
65     Summon c(12,"SELAMAT","992112035621",
66         "NO 3 JALAN PC 31 TAMAN PERMATA CAHAYA");
67
68     c.addOfficer ("AHMAD", 87611);
69     a.print();
70     b.print();
71     c.print();
72     c.removeOfficer();
73
74     return 0;
75 }

```

- a. Describe the two approaches to implement the concept of class aggregations and identify the line number of code in Program 6.2 which implement these approaches.

Name of the aggregation approach	Description of the approach	The line number that implements the aggregation approach

- b. Based on Program 6.2, draw the UML diagram showing all the classes and the relationship between them.
- c. What is the output of Program 6.2?

3. Given Program 6.3 as shown below. Answer questions (a) and (b) following it.

```

1  //Program 6.3
2  #include <string>
3  #include <iostream>
4  using namespace std;
5
6  class Name
7  {
8      private:
9          string firstname, lastname;
10
11     public:
12         Name(string fname, string lname)
13         {
14             firstname = fname;
15             lastname = lname;
16         }
17
18         string getFullName()
19         {
20             return firstname + " " + lastname;
21         }
22     };
23
24     class Lecturer
25     {
26     private:
27         Name name;
28         string staffId;
29
30     public:
31         Lecturer(string fname, string lname, string sId):
32             name(fname, lname)
33         {
34             staffId = sId;

```

```

35     }
36
37     string getLecturer()
38     {
39         return name.getFullName() + "\nLecturer id : " +
40             staffId;
41     }
42 };
43
44 class Department
45 {
46     private:
47         Lecturer *lecturerDepart;
48
49     public:
50         Department(Lecturer *lectDepart)
51         {
52             lecturerDepart = lectDepart;
53         }
54
55         void printDepartment()
56         {
57             cout << "Lecturer name: " <<
58                 lecturerDepart->getLecturer() << endl;
59         }
60 };
61
62 int main()
63 {
64     Lecturer *lect = new Lecturer("Abdullah", "Hamid", "124");
65     Department department(lect);
66     department.printDepartment();
67
68     return 0;
69 }

```

- a. Based on Program 6.3 given above, draw the UML class diagram that shows the relationship between classes.
- b. What is the output of Program 6.3?
- c. Write the class definitions that contain aggregation relationship for the following classes as depicted in Figure 6.1.

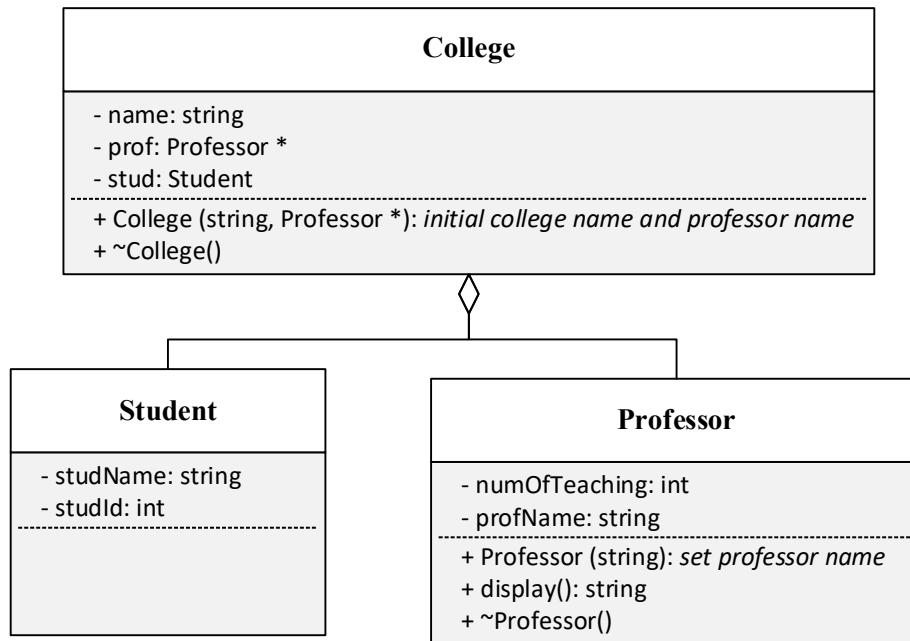


Figure 6.1: Aggregation relationship

4. Given Program 6.4 as shown below. Answer questions (a) and (b) following it.

```

1 //Program 6.4
2 #include <iostream>
3 using namespace std;
4
5 class classA
6 {
7     int val1;
8     string val2;
9
10    public:
11        classA(int val1, string val2)
12        { this->val1 = val1;
13          this->val2 = val2; }
14
15        int getValue() { return val1 * 100; }
16        string getStringA() { return val2; }
17 };
18
19 class classC
20 {
21     string value1;
22     string value2;
23
24    public:
25        classC(string value1, string value2)
26        { this->value1 = value1;
27          this->value2 = value2; }
28
29        string getStringC() { return value1 + ' ' + value2; }
30 };
  
```

```

31
32 class classB
33 {
34     string valA;
35     int valB;
36     classA *valC;
37     classC valD;
38
39     public:
40     classB(string valA, int valB, classA *C): valD("Good",
41         "Luck")
42     { this->valA = valA;
43       this->valB = valB;
44       valC = C; }
45
46     void print()
47     { cout << "In class A: " << valC->getStringA()
48         << " " << valC->getValue() << endl
49         << "In class C: " << valD.getStringC() << endl
50         << "In class B: " << valA << " " << valB * 100; }
51 };
52
53 int main()
54 {
55     classA *obj1 = new classA(50, "My target");
56     classB obj2("For You", 10, obj1);
57     obj2.print();
58     return 0;
59 }

```

- a. Based on Program 6.4 above, draw the UML class diagram that shows the relationship among classes.
 - b. What is the output of this program?
5. Consider the Program 6.5 and answer questions (a) and (b).

```

1 //Program 6.5
2 #include <iostream>
3 using namespace std;
4
5 class Instructor
6 {
7     private:
8         string instructorName;
9         string officeNum;
10
11     public:
12         Instructor(string n, string o)
13         { set(n, o); }
14
15         void set (string n, string o)
16         { instructorName = n;
17           officeNum = o; }

```

```

18
19     void print() const
20     { cout << "\n Instructor name: " << instructorName;
21       cout << "\nOffice number: " << officeNum << endl; }
22 };
23
24 class TextBook
25 {
26     private:
27         string title;
28         string author;
29
30     public:
31         TextBook(string t, string a)
32         { set(t, a); }
33
34         void set(string t, string a)
35         { title = t;
36           author = a; }
37
38         void print() const
39         { cout << " \nTitle: " << title << endl;
40           cout << "Author: " << author << endl; }
41 };
42
43 class Exam
44 {
45     private:
46         string examName;
47         string date;
48
49     public:
50         Exam()
51         { set("", ""); }
52
53         Exam(string n, string d)
54         { set(n, d); }
55
56         void set (string n, string d)
57         { examName = n;
58           date = d; }
59
60         void print() const
61         { cout << "\nExam name: " << examName << endl;
62           cout << "Date: " << date << endl; }
63 };
64
65 class Course
66 {
67     private:
68         string courseName;
69         Instructor *instructor;
70         TextBook *textbook;

```

```

71     Exam exam;
72
73     public:
74         Course(string n, string d, string c, Instructor *I,
75             TextBook *T): exam(n, d)
76         { courseName = c;
77           instructor = I;
78           textbook = T; }
79
80         void print() const
81         { cout << "Course name: " << courseName;
82           instructor->print();
83           textbook->print();
84           exam.print(); }
85     };
86
87     class Department
88     {
89     private:
90         string depName;
91         Instructor *instructor;
92
93     public:
94         Department(string n, Instructor *I)
95         { depName = n;
96           instructor = I; }
97
98         void print() const
99         { cout << "\nDepartment name: " << depName;
100           instructor->set("Amir Hamzah", "N28-301");
101           instructor->print(); }
102     };
103
104     int main()
105     {
106         Instructor *myInstructor = new Instructor("Noraminah
107             Hassan", "N28A-512");
108         TextBook *myText = new TextBook("Introduction to C++",
109             "Daniel Liang");
110         Department myDepart("Software Engineering", myInstructor);
111         Exam myExam("Final Exam", "05 January 2018");
112         Course myCourse("Test 1", "07 November 2017", "Programming
113             Technique II", myInstructor, myText);
114
115         myCourse.print();
116         myDepart.print();
117         myText->set("Starting Out with C++", "Gaddis");
118         myInstructor->print();
119         myText->print();
120         myExam.print();
121
122         return 0;
123     }

```


- a. Based on Program 6.5, draw the UML diagram showing all the classes and the relationship between them.
 - b. What is the output of this program 6.5?
6. Using Object-oriented Programming approach, develop a C++ program for a shipping company to manage their inventory of ships. The company keeps the following information about each ship:
- a. The name of the ship
 - b. The year that the ship was built
 - c. The address that the ship was registered at. It includes the country where the ship was registered and the registrar office that keeps the registration record.

The UML class diagram for the inventory system is given as in Figure 6.2. In class **Address**, the function **set** is a mutator used for specifying the member variables, **registrar** and **country**, with values obtained from the function's arguments, while the functions **getRegistrar** and **getCountry** are accessors.

As for the class **Ship**, the function **read** is a mutator that specifies all the member variables, **name**, **yearMade** and **address**, with values obtained from the keyboard. The function **print** on the other hand, is used to print the member variables.

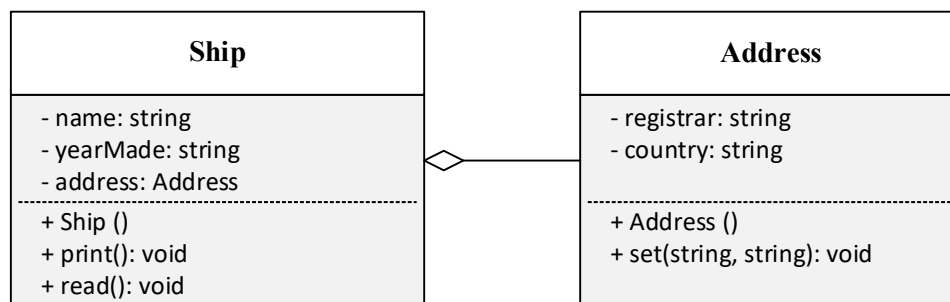


Figure 6.2: Class diagram for the ship inventory system

The implementation of your program must be based on the class diagram and specifications given above. Also, your program should use a dynamically allocated array to store the list of ships. Furthermore, the program should provide a menu-driven interaction for the user to use the program. It should allow the user to add the record for a ship one at a time. It should also provide an operation to display the number of ships and the information of each ship. Figure 6.3 illustrates an example run of the program. Note that the **bold texts represent** user inputs.

```

===== MENU =====
1. Add a ship
2. Display ships
3. Exit

Choose an operation => 1
  
```

```
<<< Enter the information of the ship >>>
```

Ship Name: **Super Gemilang**

Year Built: **2010**

The address the ship was registered:

Registrar Office: **Malaysian Shipping Council**

Country: **Malaysia**

Press any key to continue . . .

```
===== MENU =====
```

1. Add a ship
2. Display ships
3. Exit

Choose an operation => **1**

```
<<< Enter the information of the ship >>>
```

Ship Name: **Orion Cruise**

Year Built: **2000**

The address the ship was registered:

Registrar Office: **Shipping Authority Board**

Country: **Singapore**

Press any key to continue . . .

```
===== MENU =====
```

1. Add a ship
2. Display ships
3. Exit

Choose an operation => **2**

```
<<< Inventory of ships >>>
```

Total ship: 2

```
==== Ship List ====
```

Ship Name: Super Gemilang

Year Built: 2010

Registered at:

Malaysian Shipping Council, Malaysia

Ship Name: Orion Cruise

Year Built: 2000

Registered at:

Shipping Authority Board, Singapore

Press any key to continue . . .

```
===== MENU =====
```

1. Add a ship
2. Display ships
3. Exit

Choose an operation => **3**

Figure 6.3: Example run

7. A faculty in a private university wishes to computerise its student records system. The record of each student includes his or her name and the program he or she enrolls. The faculty offers several programs at postgraduate level such as “Master of Business Administration”, “PhD of Social Science”, and many more. Regardless of the program or degree, each student is appointed with a lecturer as his or her academic advisor. The role of the advisor is to guide the student on academic matters. As for the postgraduate programs either master or PhD degrees, they are conducted in fully research-based. Each postgraduate student has to have a research project and a lecturer to supervise his or her project.

Based on the given problem, answer the following questions:

- Draw the UML class diagram for the above problem. Your design has to include the classes and their attributes and methods accordingly, as well as relationships between the classes. Each class has to provide a constructor, mutators, and assessors.
- Then, write the C++ code to implement the design. Your implementation should apply object-oriented programming concepts including data hiding, composition and/ or aggregation.
- Next, utilize the classes to store a list of postgraduate students. You need to use dynamic arrays for the list and fill it in with data read from an input file. Figure 6.4 shows the example of input file containing the list of postgraduate students. The first line in each file indicates the number of students the file contains. Following that is the record of a student in which each attribute is arranged in a line. The student records are separated by blank lines. Finally, print all the students from the arrays into another text file. Figure 6.5 shows an example of the output file.

50	}	<i>The number of students</i>
Bakar Bin Malik	}	<i>Student's name and program</i>
Master of Social Science	}	
Dr. Rohaya Qamarool	}	<i>Academic advisor</i>
Prof. Dr. Kamil Ahmad	}	<i>Project supervisor</i>
The Effects of Social Media	}	<i>Research project</i>
Ruby Gabriella		
PhD of Business		
Prof. Dr. Kamil Ahmad		
Prof. Dr. Saidi Abdullah		
Adaptive Human Capital Management		

(due to limited spaces, the remaining student records are not shown in this figure)

Figure 6.4: An example of input file named “pgstudents.txt”, for the list of postgraduate student records

Note that, the texts in *italic* are given to describe the fields accordingly. Also, only two records from the input file out of 50 are shown here due to limited spaces.

THE LIST OF POSTGRADUATE STUDENTS		
No	Name	Supervisor Project
1	Bakar Bin Malik	Prof. Dr. Kamil Ahmad The Effects of Social Media
2	Ruby Gabriella	Dr. Saidi Abdullah An Adaptive Human Capital Management
..		
		(students no 3 to 49 go here)
..		
50	Zul Bin Hashim	Prof. Dr. Kamil Ahmad Artificial intelligence for robots

Figure 6.5: An example of the output file containing the lists of postgraduate students
Note that only some students are shown Figure 6.5 due to limited spaces.

8. Consider the class diagram in Figure 6.6 which shows the data model for a car rental company. Note that the company has set the rule that each customer can only rent one car at a time. Based on the class diagram, write a C++ program which performs the following tasks:

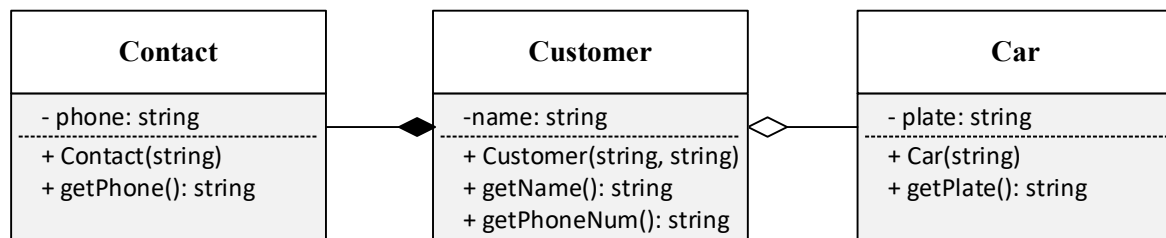


Figure 6.6: Class diagram for a car rental service

- Implement all the three classes with the given attributes and operations. Note that, the purpose of each operation is as the name implies.
- Test the classes by creating an object of **Car** and an array of customers with the following data:

Customer's Name	Phone Number	Rented Car Plate
Ahmad Kamal	015-75769800	JSQ245
Siti Nurdiana Abdullah	014-8889900	

Note that, the column “**Rented Car Plate**” for the second customer is empty because she does not rent any car at the moment.

- Print the array of customers onto the screen. The screen output should look like as in Figure 6.7.

```

Customer's Name: Ahmad Kamal
Phone Number: 015-75769800
Rented Car   : JSQ245

Customer's Name: Siti Nurdiana Abdullah
Phone Number: 014-8889900
Rented Car   :
  
```

Figure 6.7: Screen output

9. Write a complete C++ program for the Person and Cat classes based on the Figure 6.8 and the information provided.

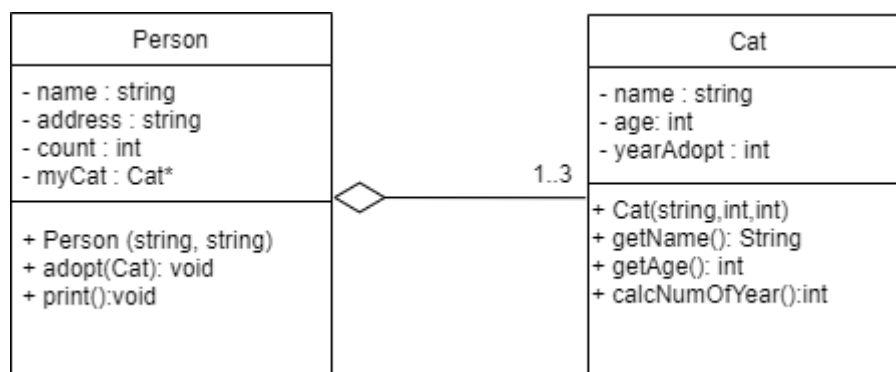


Figure 6.8: Class diagram

- a. The Person class consists of the following members:
- A constructor that accepts the person's name and address as arguments. These values should be assigned to the object's name and address member variables.
 - adopt function: This function is used to add objects from the Cat class to myCat array. The objects added to array refers to the cats adopted by person.
 - print function: This function will display the person information along with his adopted cats and number of years he has adopted the cat(s).
- b. The Cat class consists of the following members:
- A constructor that will initialize all the member attributes to the values received as arguments.
 - getName function: This function will return a value of name member attributes.
 - getAge function: This function will return a value of age of the cat adopted.
 - calcNumOfYear function: This function will calculate number of years one cat being adopted.